

PROTOTYPE · NOT AN OFFICIAL LPU SYSTEM

Research Intelligence & Collaboration Hub

Lovely Professional University (LPU) — Project Documentation

135

API OPERATIONS

39

TABLES

21

MIGRATIONS

24

ADRS

639

TESTS

23

MODULES

Disclaimer. This is a proposed prototype research platform and is **not an official LPU system** unless formally adopted or authorized. It is not connected to any real LPU system or data. Every person, project, publication, facility and funding record it displays is **fictional demo data**.

Repository	github.com/UtkarshMehta7/lpu-resource-hub
Live application	lpu-research-hub.netlify.app
API	lpu-resource-hub.onrender.com
Database	Neon (PostgreSQL 18, serverless)
Licence	MIT
Cost to run	₹0 — free tiers and locally-run open-source ML only

Contents

1. Problem, goal and scope
2. System architecture
3. Technology stack
4. Data model and entity relationships
5. Roles, permissions and account provisioning
6. Feature catalogue — what the platform does
7. Workflows and state machines
8. Security model
9. AI and recommendation engine
0. Testing strategy and quality gates
1. Deployment architecture
2. Design decisions (ADR index)

3. Known limitations and future work

1. Problem, goal and scope

1.1 The problem

Inside a large university, research capability is invisible across departmental boundaries. Faculty expertise, ongoing projects and available laboratory equipment are known locally and almost nowhere else. Three costs follow directly:

- **Collaborators never meet.** Two researchers working on adjacent problems in different schools have no mechanism to discover one another.
- **Students cannot find research.** Opportunities circulate by word of mouth, so access depends on who a student already knows.
- **Resources idle and deadlines pass.** Equipment goes under-used because nobody outside the owning department knows it exists; funding calls are missed for want of a reminder.

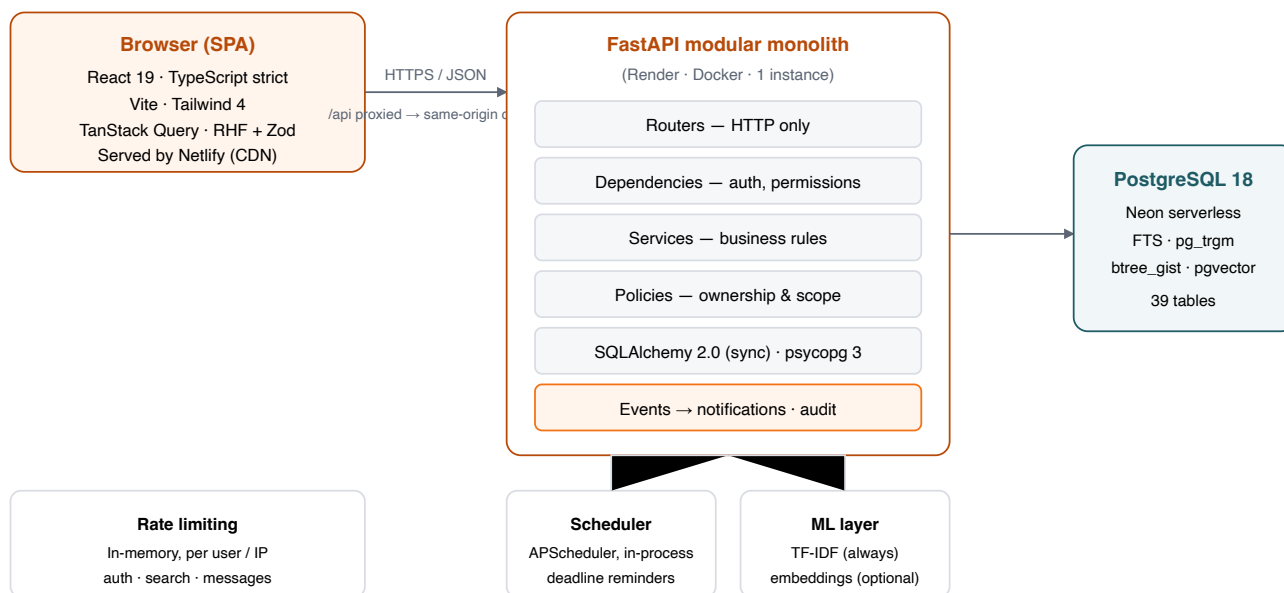
1.2 The goal

One platform where research capability is discoverable, collaboration is initiated and continued in the open, shared facilities are bookable, and funding deadlines announce themselves — with every action governed by university hierarchy rather than by a free-for-all.

1.3 Scope and constraints

Constraint	Consequence for the design
Zero cost	No paid APIs, no mandatory cloud services. Machine learning runs locally (scikit-learn, sentence-transformers). Docker is optional.
PostgreSQL only	Full-text search, trigram fuzzy matching, exclusion constraints and vector similarity are all done in the database; no Elasticsearch, no vector service.
No message broker	In-memory rate limiting, an in-process scheduler and synchronous domain events — each behind an interface so it can be replaced.
Fictional data only	The platform is a prototype. It must never present fabricated records as real university data, and says so on every page.

2. System architecture



Boundaries are enforced in code: routers contain no business logic; services never import FastAPI; each workflow's state transitions live in exactly one table. The ML layer can be extracted later.

Figure 1 — System architecture. A modular monolith with clear internal seams, chosen so a single free instance can run the whole platform.

2.1 Why a modular monolith

Microservices would multiply deployment cost and operational surface for a project that must run at zero cost on one instance. The seams that matter — router / service / policy / repository, and a pluggable recommendation strategy — are enforced inside one deployable, so the ML layer or any module can be extracted later without re-architecting.

2.2 Backend module map

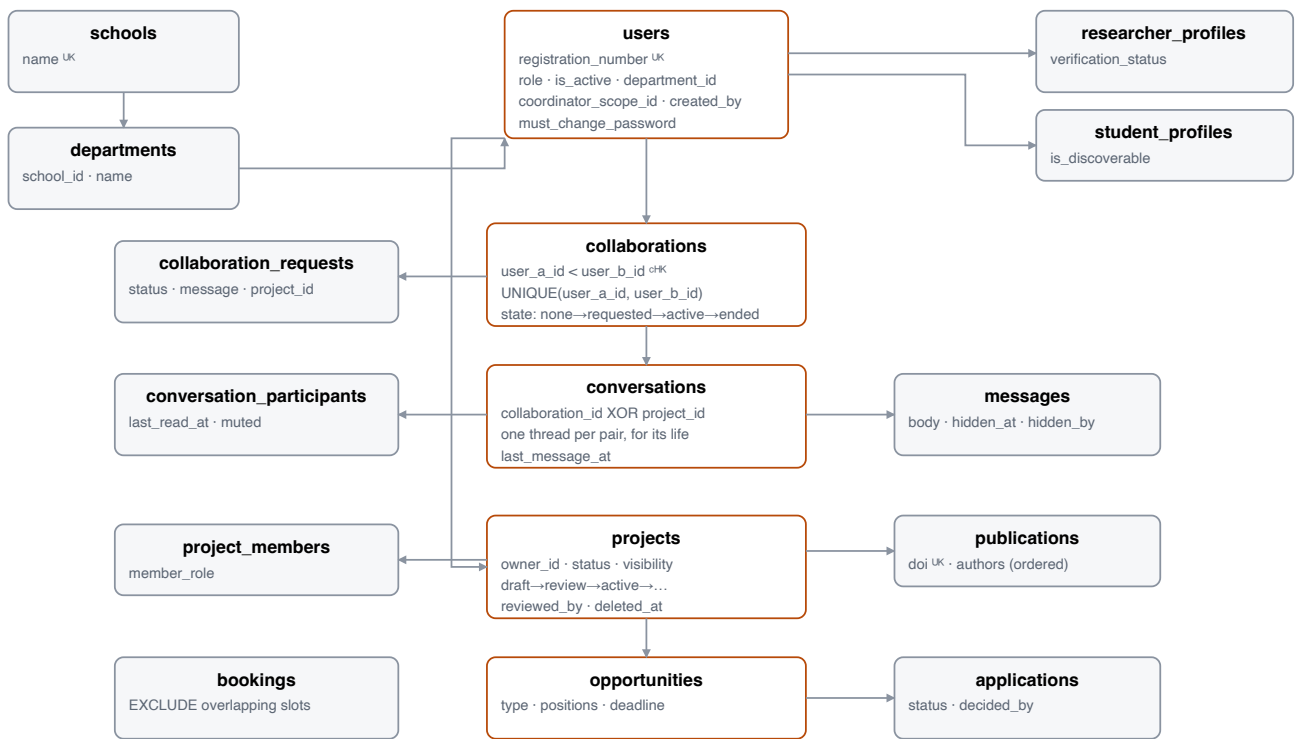
23 modules under `backend/app/modules/`, each with router, schemas, models, service and (where ownership matters) policies.

Group	Modules
Identity	auth, users, profiles, admin
Discovery	researchers, taxonomy, search, recommendations
Research work	projects, publications, opportunities, applications
Connection	collaborations, messages, notifications
Resources	facilities, bookings, funding
Governance	audit, reports, analytics, health

3. Technology stack

Layer	Technology	Why this choice
Frontend	React 19, TypeScript (strict), Vite, Tailwind CSS 4, React Router 7, Axios, TanStack Query, React Hook Form + Zod, Recharts	Strict typing catches schema drift at compile time; TanStack Query removes hand-written cache and polling code.
Backend	Python 3.12, FastAPI, Pydantic v2, pydantic-settings, SQLAlchemy 2.0 (sync), psycopg 3, Alembic	Pydantic gives request/response validation and OpenAPI for free. Sync SQLAlchemy is simpler to reason about and test; FastAPI runs it in a threadpool.
Database	PostgreSQL 18 with <code>pg_trgm</code> , <code>btree_gist</code> , <code>pgvector</code>	One engine provides relational integrity, full-text search, fuzzy matching, double-bookings prevention and vector similarity.
Security	Argon2id, PyJWT, rotating refresh tokens, CSP and security headers	Argon2id is the current password-hashing recommendation; token rotation with reuse detection limits the value of a stolen token.
AI / ML	scikit-learn (TF-IDF), sentence-transformers (optional), pgvector	Runs locally at no cost. The semantic layer is optional and degrades to lexical matching if absent.
Quality	ruff, mypy --strict, pytest · ESLint, Prettier, tsc, Vitest, Playwright	Static analysis at maximum strictness; tests at unit, API and browser level.
Operations	GitHub Actions, Docker (optional), Neon, Render, Netlify	Five CI jobs gate every push; all three hosts have free tiers sufficient for a prototype.

4. Data model and entity relationships



Core entities shown. 39 tables in total — taxonomy (skills, research_areas, aliases, suggestions), facilities/equipment, funding, notifications, saved_items, audit_logs and embeddings omitted for clarity.

Figure 2 — Entity relationships, core domain. Orange entities are the spine of the platform.

4.1 Database conventions

- UUID primary keys via `gen_random_uuid()`; `created_at` / `updated_at` as `timestampz`.
- PostgreSQL enums for workflow statuses; CHECK constraints for invariants that must never be violated.
- Every schema change is a reviewed Alembic migration with a sequential id. `create_all()` is never used.
- Deterministic constraint naming, so migrations are readable and reversible.

4.2 Constraints that carry real rules

Constraint	What it makes impossible
<code>UNIQUE(user_a_id, user_b_id)</code> with <code>CHECK (user_a_id < user_b_id)</code>	Two people having more than one collaboration relationship, in either direction. Duplicates are impossible in the database rather than guarded in code.
<code>EXCLUDE USING gist</code> on bookings	Two approved bookings overlapping on the same equipment. The database refuses; no application race can produce a double-booking.
Partial <code>UNIQUE</code> on notifications <code>dedupe_key</code>	The same reminder being sent twice, however often the scheduler runs.
<code>CHECK</code> exactly-one-of on conversations and tag aliases	A thread belonging to both a collaboration and a project, or to neither.
<code>UNIQUE</code> on <code>registration_number</code> , case-normalised	Two accounts sharing a university identity.

5. Roles, permissions and account provisioning

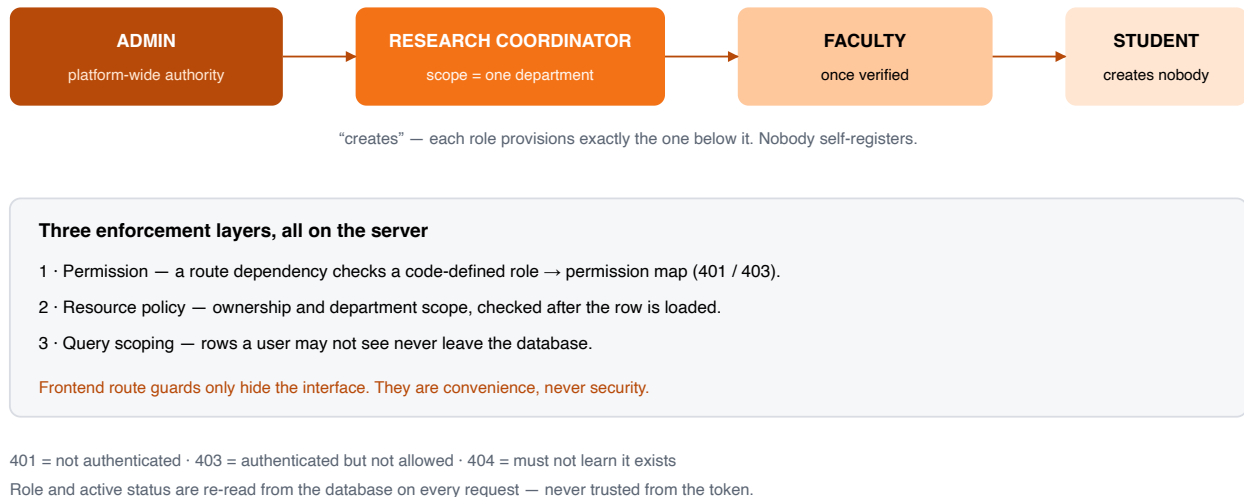


Figure 3 — Role hierarchy and the three layers of authorization.

5.1 Account provisioning

There is **no public registration**. `POST /auth/register` does not exist, and a test asserts the OpenAPI document contains no such path. Accounts descend the institutional hierarchy: an administrator appoints research coordinators, a coordinator provisions the faculty of the department they oversee, and a faculty member enrolls their own students.

The new account's role is **derived from the creator** and never read from the request body — there is no client-supplied role to forge. Each new user receives a temporary password they must replace before they can use anything else.

5.2 Administrator handover

Promoting somebody to administrator requires two people. A six-digit code is generated, hashed with SHA-256, and delivered to the *target's* notification inbox — not the requester's. It expires in ten minutes, is single-use, and allows five attempts. Administration also has its own sign-in page, and the server refuses the wrong entrance for each role. That check runs *after* the password is verified, so the administration page can never become an oracle telling an attacker which registration numbers belong to administrators.

5.3 Permission matrix (abridged)

Capability	Student	Faculty	Coordinator	Admin
Browse directory, search, recommendations	✓	✓	✓	✓
Apply to opportunities	✓	collaboration openings only	—	—
Create projects, publications, openings	—	✓ (verified)	✓	—
Create accounts	—	students, own dept	faculty, own scope	coordinators / any (audited override)
Delete accounts	—	students, own dept	faculty + students, own scope	anyone but themselves
Verify researchers, review projects, approve bookings	—	—	✓ own department	✓ platform-wide
Moderate reports, read analytics	—	—	✓ scoped	✓ platform
Read audit log, manage settings	—	—	—	✓

6. Feature catalogue

6.1 Identity and onboarding

- **Sign in with the LPU registration number** (UMS-style); email is optional contact information, never a credential.
- **Two separate entrances** — administration and the main platform — enforced server-side.
- **Forced password replacement** on first sign-in; the application is locked until it is done.
- **Onboarding wizard** — profile basics, skills, research areas — with a stored completeness flag.
- **Role-aware profiles**: students declare programme, year and discoverability; researchers declare designation, availability and links.

6.2 Discovery

- **Researcher directory** with full-text and typo-tolerant (trigram) search, filtered by department, research area, skill and availability.
- **Search by registration number** — matched as a case-insensitive prefix, not through the language analyser, because an identifier is not prose.
- **Opt-in student discovery**. Students are invisible to search unless they choose otherwise; only public profile fields are ever exposed.
- **Unified search** across researchers, projects, publications and opportunities.
- **Natural-language search** — sentence embeddings in pgvector fused with full-text ranking; degrades cleanly to lexical search when the optional model is not installed.
- **Command palette** (⌘K / Ctrl+K) reaching all 28 destinations, offering only pages the signed-in role may open.

6.3 Research work

- **Projects** with a reviewed lifecycle, team members, research areas, skills and visibility rules.
- **Publications** with ordered internal and external author lists, DOI de-duplication and project links.
- **Opportunity board** — research assistant, thesis, internship and collaboration openings, with positions and deadlines.
- **Applications** with a status timeline, applicant review and automatic addition to the project team on acceptance.

6.4 Connection

- **Collaboration relationships** — one per pair of people, in either direction, with a request/accept/decline/cancel/end lifecycle.
- **Conversation threads** scoped to an accepted collaboration or a project team. There is no open inbox and no endpoint that creates a thread: threads appear as a consequence of agreement.

- **One thread per pair for its life** — collaborating again reopens the same conversation instead of splitting two people's history.
- **Notifications** from domain events, with deduplication: twenty messages produce one notice until it is read.
- **Deadline reminders** at seven days and one day, from an in-process scheduler.

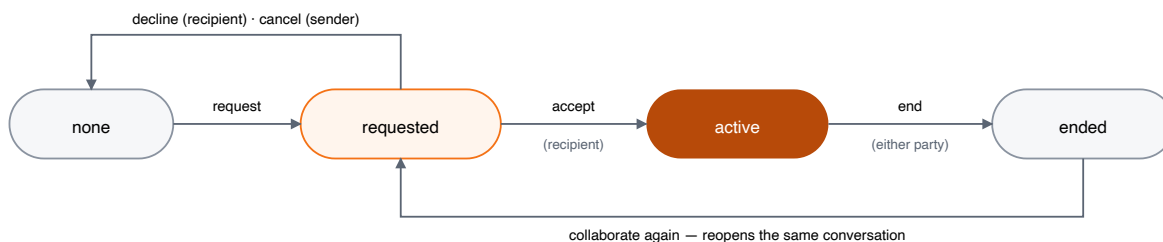
6.5 Resources

- **Facility and equipment catalogue** with a week calendar.
- **Booking requests and approvals**, with double-booking prevented by a database exclusion constraint rather than by application logic.
- **Funding calls** (fictional) that can be saved, with deadline reminders.

6.6 Governance

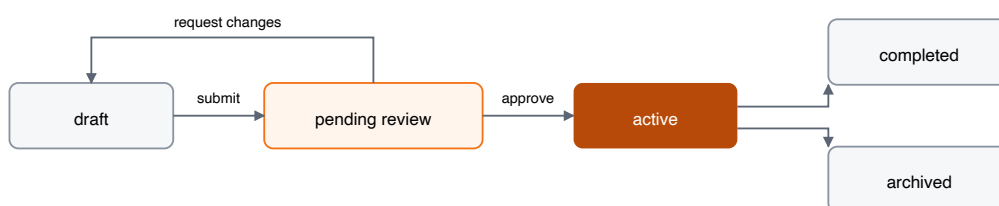
- **Faculty verification queue** — department-scoped for coordinators, platform-wide for administrators; nobody may verify their own profile.
- **Project review queue**; a reviewer may never approve their own project.
- **Content reports and moderation**, able to hide content without destroying it.
- **Append-only audit log** for every sensitive action, readable by administrators.
- **Scoped analytics** — coordinators see their department, administrators the platform — plus a collaboration network graph.
- **Account deletion** down the hierarchy, with the cascade counted and shown before anything is destroyed.

7. Workflows and state machines



A thread is writable only while the relationship is active, and readable always: what was said still happened.
It takes two people to start a collaboration and one to end it — requiring agreement to stop would mean nobody could leave.

Figure 4 — Collaboration state machine. Defined once, in a single transition table, and enforced in the service layer.



Only an ACTIVE project accepts applications. A coordinator may never review their own project.

Figure 5 — Project review lifecycle.

7.1 Other workflows

Workflow	States	Who may transition
Researcher verification	unverified → pending → verified / rejected	Coordinator (own department) or administrator; never oneself
Opportunity	draft → open → closed / filled	Creator; auto-fills when positions are taken
Application	submitted → under review → accepted / rejected / withdrawn	Owner decides; applicant may withdraw
Booking	requested → approved / rejected / cancelled	Coordinator approves; database prevents overlap
Content report	open → dismissed / actioned	Coordinator or administrator; may hide the target

8. Security model

Control	Implementation
Password storage	Argon2id with per-password salt. Passwords are never logged, returned or comparable in plaintext.
Session	15-minute JWT access token held in memory only — never in localStorage, so an XSS cannot read it at rest.
Refresh	Opaque 7-day token in an <code>httpOnly; Secure; SameSite</code> cookie scoped to <code>/api/v1/auth</code> , stored as a SHA-256 hash, rotated on every use. Reuse of a retired token revokes the entire family.
CSRF	Every browser request carries <code>X-Requested-With</code> ; the cookie is same-origin via the Netlify proxy.
Authorization	Three server-side layers (§5). Role and active status re-read from the database on every request.
Information disclosure	404 — not 403 — wherever a user must not learn a resource exists: another department's queue, a conversation they are not in, an administrator's registration number.
Rate limiting	Per-user and per-IP budgets on sign-in, search, collaboration requests and messages.
Transport and headers	HSTS, CSP, <code>X-Content-Type-Options</code> , frame denial. API docs disabled in production.
Secrets	Never committed. All configuration via environment variables; <code>.env.example</code> holds placeholders only. CI runs a secret scan and dependency audit on every push.
Audit	Append-only log of every sensitive action — role changes, approvals, deletions, administrative overrides — recording actor, entity, before/after and IP.

9. AI and recommendation engine

Recommendations are **explainable by design**. Every suggestion carries the reasons it was made, because a black box that cannot justify itself will not be trusted by the researchers it is advising.

Signal	Method	Weight
Skill overlap	Set intersection against the viewer's declared skills	Highest
Research-area overlap	Taxonomy match, including two-level parent relationships	High
Textual similarity	TF-IDF vectors with cosine similarity (scikit-learn), computed locally	Medium
Semantic similarity	Sentence-transformer embeddings in pgvector, fused with full-text rank	Optional
Recency and availability	Open positions, deadlines, stated availability	Tie-break

A user with a sparse profile receives a **cold-start** response that says so plainly and offers the newest relevant items rather than pretending to a confidence it does not have. The engine is a pluggable strategy, and an offline evaluation script measures ranking quality.

10. Testing strategy and quality gates

454

BACKEND TESTS

185

FRONTEND TESTS

14

E2E FLOWS

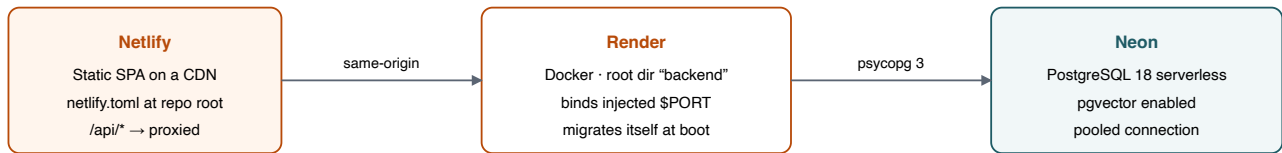
5

CI JOBS

Level	Tooling and approach
Static analysis	<code>ruff</code> (lint + format), <code>mypy --strict</code> across 159 files. No <code>any</code> , no blanket <code>type: ignore</code> , no disabled rules to make a check pass.
Backend	<code>pytest</code> against a real PostgreSQL database — never SQLite — with each test rolled back. Behaviour is tested through the API, not through mocks.
Authorization	A generated matrix asserts the exact status code for every endpoint × role combination, so a new endpoint cannot quietly be left unguarded.
Frontend	Vitest with Testing Library, queried by role and label as a user would find them.
Browser	Playwright drives 14 flows end to end against a seeded database — project submission through review, application, booking, collaboration, account provisioning and deletion.
CI	Five jobs per push: backend, backend with the ML extra, frontend, end-to-end, and security (dependency audit + secret scan).

A principle learned the hard way: passing tests are not a working screen. A one-time administrator code was delivered correctly and rendered invisibly, with every test green, because a display function had no case for it. New interface work is therefore driven in a real browser before it is called done.

11. Deployment architecture



Why the API is proxied rather than called cross-origin

The refresh token lives in an httpOnly cookie. Proxying /api through Netlify keeps it first-party, so SameSite=lax applies and the session survives a page refresh without relaxing cookie policy for third-party contexts — which would weaken CSRF protection for every user.

Figure 6 — Deployment topology. Every component sits on a free tier.

11.1 Release process

A push to `main` deploys. The API applies outstanding Alembic migrations at startup and, if that fails, starts *degraded* and reports the reason on `/health/ready` rather than refusing to serve — a failed deploy leaves the previous image running and the fault invisible. Data-rewriting migrations are still run deliberately from a workstation before the code that needs them merges.

12. Design decisions (ADR index)

Twenty-four architecture decision records in `docs/adr/`. The most consequential:

ADR	Decision	Why it mattered
0015	Sign in with the registration number	Matches how the university already identifies people; email becomes optional contact data.
0017	Deployment and hardening	Free-tier topology, security headers, cookie strategy.
0018	Department membership	Self-declared until verification locks it; an administrator can always correct it.
0019	Provisioning hierarchy	Removed public registration; the role is derived from the creator, never sent by a client.
0020	Identity on screen, derived coordinator scope	The UID appears wherever a person is named; a coordinator's scope follows their department automatically, because one with no scope silently oversaw nothing.
0021	Account deletion	Deletion down the hierarchy, with the cascade counted and shown first — 21 of 31 foreign keys cascade.
0022	Scoped conversation threads	Chat exists only where a relationship already does; polling rather than WebSockets, with the reasoning recorded.
0023	Collaboration belongs to a pair	Keying the relationship to a request allowed duplicates and split two people's history; the pair is now unique in the database.
0024	Migrate at startup	Code repeatedly reached production ahead of its migration. The instance now heals itself and reports failure honestly.

13. Known limitations and future work

Limitation	Reasoning and the path forward
Messaging polls rather than pushes	Deliberate: synchronous SQLAlchemy, no Redis, and a free instance that sleeps. A 10-second poll that pauses when the tab is hidden is indistinguishable from real time at this scale. Server-Sent Events are the upgrade path — one endpoint, no schema change.
Semantic search optional in production	The sentence-transformer model exceeds a free instance's memory. Embeddings can be precomputed offline, or the paid tier used.
Single application instance	Rate limiting and the scheduler are in-process. Horizontal scaling needs shared state — both already sit behind interfaces.
Notifications are in-app only	Email or push would need a provider and therefore a cost.
All data is fictional	Integration with real university systems is deliberately out of scope for a prototype.

13.1 Natural next steps

1. Institutional single sign-on in place of local passwords.
2. Email notifications and calendar export for bookings and deadlines.
3. Co-authorship and citation graphs from the publication data already stored.

4. Generated OpenAPI TypeScript types, removing the hand-written frontend mirror of each schema.
5. An expertise heat-map across schools, using the taxonomy already in place.

Prototype notice. Not an official Lovely Professional University system unless formally adopted or authorized. Not connected to any real LPU system or data. All people, projects, publications, facilities and funding records shown are fictional demo data.